

Navigation in Flutter

Navigation in Flutter

Eng: *Ruqaih Hussein Salman*

2025-2026

Navigating Between Screens

Flutter already provides built-in navigator APIs that can help us navigate between screens (routes) and show dialogs, snack bars, bottom sheets without installing a third-party library.

Flutter Navigation and Routing

Navigation and routing are some of the core concepts of all mobile application, which allows the user to move between different pages. We know that every mobile application contains several screens for displaying different types of information. **For example**, an app can have a screen that contains various products. When the user taps on that product, immediately it will display detailed information about that product.

In Flutter, the screens and pages are known as **routes**, and these routes are just a widget. In Android, a route is similar to an **Activity**, whereas, in iOS, it is equivalent to a **ViewController**.

In any mobile app, navigating to different pages defines the workflow of the application, and the way to handle the navigation is known as **routing**. Flutter provides a basic routing class **MaterialPageRoute** and two methods **Navigator.push()** and **Navigator.pop()** that shows how to navigate between two routes. The following steps are required to start navigation in your application.

- **Step 1:** First, you need to create two routes.
- **Step 2:** Then, navigate to one route from another route by using the `Navigator.push()` method.
- **Step 3:** Finally, navigate to the first route by using the `Navigator.pop()` method.

Let us take a simple example to understand the navigation between two routes:

Create two routes

Here, we are going to create two routes for navigation. In both routes, we have created only a **single button**. When we tap the button on the first page, it will navigate to the second page. Again, when we tap the button on the second page, it will return to the first page. The below code snippet creates two routes in the Flutter application.

```
import 'package:flutter/material.dart';
class FirstRoute extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('First Route'),
      ),
      body: Center(
        child: ElevatedButton(
          child: Text('Open route'),
          onPressed: () {
```

```

        // Navigate to second route when tapped.
      },
    ),
  ),
);
}
}

class SecondRoute extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Second Route"),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            // Navigate back to first route when tapped.
          },
          child: Text('Go back!'),
        ),
      ),
    );
  }
}

```

Navigate to the second route using `Navigator.push()` method

The `Navigator.push()` method is used to navigate/switch to a new route/page/screen. Here, the **`push()`** method adds a page/route on the stack and then manage it by using the **Navigator**. Again we use `MaterialPageRoute` class that allows transition between the routes using a platform-specific animation. The below code explain the use of the `Navigator.push()` method.

```

// Within the `FirstRoute` widget
onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => SecondRoute()),
  );
}

```

Return to the first route using `Navigator.pop()` method

Now, we need to use **`Navigator.pop()`** method to close the second route and return to the first route. The `pop()` method allows us to remove the current route from the stack, which is managed by the `Navigator`. To implement a return to the original route, we need to update the **`onPressed()`** callback method in the `SecondRoute` widget as below code snippet:

```

// Within the SecondRoute widget
onPressed: () {
  Navigator.pop(context);
}

```

Now, let us see the full code to implement the navigation between two routes. First, create a Flutter project and insert the following code in the **main.dart** file.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    title: 'Navigation Roq App',
    theme: ThemeData(
      // This is the theme of your application.
      primarySwatch: Colors.blueGrey,
    ),
    home: FirstRoute_Roq_Ex(),
  ));
}

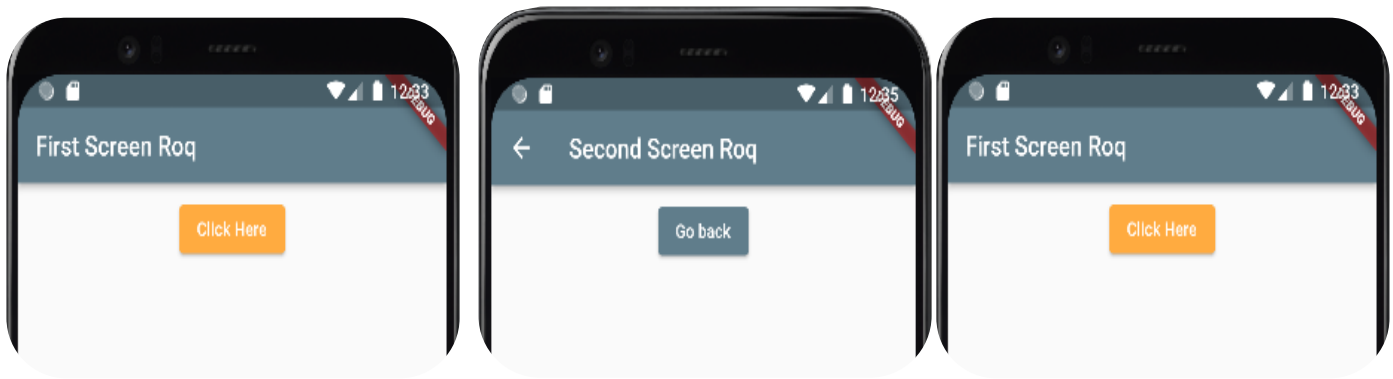
class FirstRoute_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('First Screen Roq'),
      ),
      body: Container(
        padding: EdgeInsets.all(10),
        alignment: Alignment.topCenter,
        child: ElevatedButton(
          child: Text('Click Here'),
          style:
            ElevatedButton.styleFrom(backgroundColor: Colors.orangeAccent),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => SecondRoute_Roq_Ex()),
            );
          },
        ),
      ),
    );
  }
}

class SecondRoute_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Second Screen Roq"),
      ),
      body: Container(
        padding: EdgeInsets.all(10),
        alignment: Alignment.topCenter,
        child: ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.blueGrey.shade500),
          onPressed: () {
            Navigator.pop(context);
          },
          child: Text('Go back'),
        ),
      ),
    );
  }
}
```

```
    },  
  );  
}  
}
```

Output

When you run the project in the **Android Studio**, you will get the following screen in your emulator. It is the first screen that contains only a single button. Click the button **Click Here**, and you will navigate to a second screen as below image. Next, when you click on the button **Go Back**, you will return to the first page.



Navigation with Named Routes

We have learned how to navigate to a new screen by creating a new route and manage it by using the Navigator. The Navigator maintains the stack-based history of routes. If there is a need to navigate to the same screen in many parts of the app, this approach is not beneficial because it results in **code duplication**. The solution to this problem can be removed by defining the named routes and can use the **named routes** for navigation.

We can work with named routes by using the **Navigator.pushNamed()** function. This function takes two required arguments (build context and string) and one optional argument. Also, we know about the **MaterialPageRoute**, which is responsible for page transition. If we do not use this, then it is difficult to change the page.

The following steps are necessary, which demonstrate how to use named routes.

- **Step 1:** First, we need to create two screens. The following code creates the two screens in our app.

```
import 'package:flutter/material.dart';  
  
void main() => runApp(Main_Routs_Roq());  
  
class Main_Routs_Roq extends StatelessWidget {
```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    theme: ThemeData(
      primarySwatch: Colors.blueGrey,
    ),
    debugShowCheckedModeBanner: false,
  );
}
}

//////////////////////////////////// First Page //////////////////////////////////////
class HomeScreen_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('First Screen Roq'),
      ),
      body: Container(
        padding: EdgeInsets.all(10),
        alignment: Alignment.topCenter,
        child: ElevatedButton(
          child: Text('Click Here'),
          style: ElevatedButton.styleFrom(backgroundColor: Colors.orangeAccent),
          onPressed: () {
            //
          },
        ),
      ),
    );
  }
}

//////////////////////////////////// Secreen 2 //////////////////////////////////////
class SecondScreen_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Second Screen Roq"),
      ),
      body: Container(
        padding: EdgeInsets.all(10),
        alignment: Alignment.topCenter,
        child: ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.blueGrey.shade500,
          ),
          onPressed: () {
            //
          },
          child: Text('Go back!'),
        ),
      ),
    );
  }
}
}

```

➤ **Step 2:** Define the routes.

In this step, we have to define the routes. The MaterialApp constructor is responsible for defining the initial route and other routes themselves. Here the initial route tells the start of the page and routes property defines the available named routes and widgets. The following code explains it more clearly.

```
MaterialApp(  
  title: 'Named Route Navigation',  
  theme: ThemeData(  
    // This is the theme of your application.  
    primarySwatch: Colors.blueGrey,  
  ),  
  // It start the app with the "/" named route. In this case, the app starts  
  // on the HomeScreen widget.  
  initialRoute: '/',  
  routes: {  
    // When navigating to the "/" route, build the HomeScreen widget.  
    '/': (context) => HomeScreen(),  
    // When navigating to the "/second" route, build the SecondScreen widget.  
    '/second': (context) => SecondScreen(),  
  },  
);
```

- **Step 3:** Navigate to the second screen using the Navigator.pushNamed() function.

In this step, we need to call Navigator.pushNamed() method for navigation. For this, we need to update an onPressed() callback in the build method of **HomeScreen** like below code snippets.

```
onPressed: () {  
  // Navigate to the second screen by using the named route.  
  Navigator.pushNamed(context, '/second');  
}
```

- **Step 4:** Use a Navigator.pop() function to return to the first screen.

It is the final step, where we will use Navigator.pop() method to return on the first screen.

```
onPressed: () {  
  Navigator.pop(context);  
},
```

Let us see the full code of the above explanation in the Flutter project and run it in the emulator to get the output.

```
import 'package:flutter/material.dart';  
  
void main() => runApp(Main_Routs_Roq());  
  
class Main_Routs_Roq extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      theme: ThemeData(  
        primarySwatch: Colors.blueGrey,  
      ),  
      debugShowCheckedModeBanner: false,  
    );  
  }  
}
```

```

    initialRoute: '/',
    routes: {
      // When navigating to the "/" route, build the FirstScreen widget.
      '/': (context) => HomeScreen_Roq_Ex(),
      // When navigating to the "/second" route, build the SecondScreen widget.
      '/second': (context) => SecondScreen_Roq_Ex(),
    },
  );
}
}

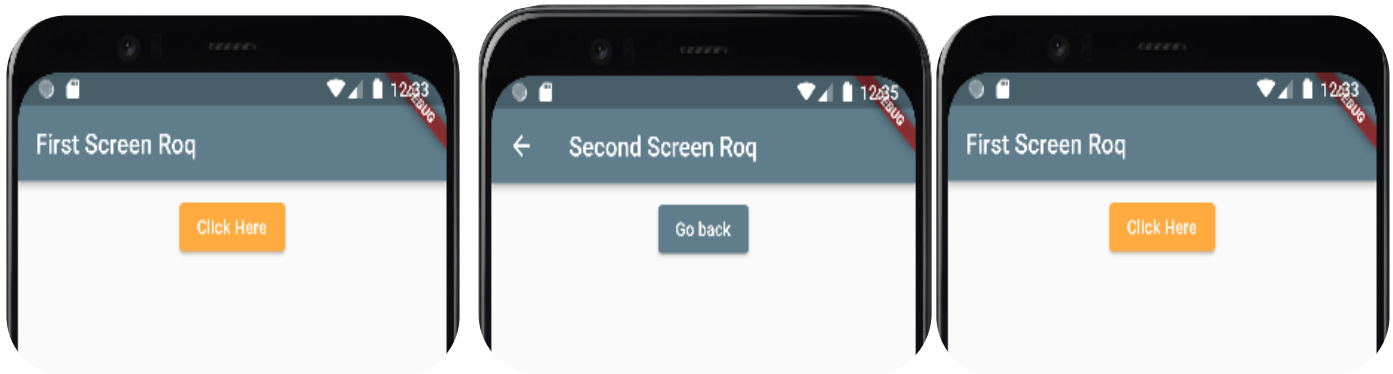
//////////////////////////////////// First Page //////////////////////////////////////
class HomeScreen_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('First Screen Roq'),
      ),
      body: Container(
        padding: EdgeInsets.all(10),
        alignment: Alignment.topCenter,
        child: ElevatedButton(
          child: Text('Click Here'),
          style: ElevatedButton.styleFrom(backgroundColor: Colors.orangeAccent),
          onPressed: () {
            Navigator.pushNamed(context, '/second');
          },
        ),
      ),
    );
  }
}

//////////////////////////////////// Secreen 2 //////////////////////////////////////
class SecondScreen_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Second Screen Roq"),
      ),
      body: Container(
        padding: EdgeInsets.all(10),
        alignment: Alignment.topCenter,
        child: ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.blueGrey.shade500,
          ),
          onPressed: () {
            Navigator.pop(context);
          },
          child: Text('Go back!'),
        ),
      ),
    );
  }
}
}

```

Output

When you run the project in the **Android Studio**, you will get the following screen in your emulator. It is the first screen that contains only a single button. Click the button **Click Here**, and you will navigate to a second screen as below image. Next, when you click on the button **Go Back**, it will return to the Home Page.



Pass arguments to a named route

The **Navigator** provides the ability to navigate to a named route from any part of an app using a common identifier. In some cases, you might also need to pass arguments to a named route. For example, you might wish to navigate to the **/user** route and pass information about the user to that route. You can accomplish this task using the **arguments** parameter of the **Navigator.pushNamed()** method. Extract the arguments using the **ModalRoute.of()** method or inside an **onGenerateRoute()** function provided to the **MaterialApp** or **CupertinoApp** constructor.

This recipe demonstrates how to pass arguments to a named route and read the arguments using **ModalRoute.of()** using the following steps:

1. Define the arguments you need to pass.
2. Create a widget that extracts the arguments.
3. Register the widget in the **routes** table.
4. Navigate to the widget.

1. Define the arguments you need to pass

First, define the arguments you need to pass to the new route. In this example, pass two pieces of data: The **title** of the screen and a **message**.

To pass both pieces of data, create a class that stores this information.

```
// You can pass any object to the arguments parameter.
// In this example, create a class that contains both
// a customizable title and message.
class ScreenArguments {
  final String name;
```

```

final int age;

ScreenArguments(this.name, this.age);
}

```

2. Create a widget that extracts the arguments

Next, create a widget that extracts and displays the `title` and `message` from the `ScreenArguments_Roq`. To access the `ScreenArguments_Roq`, use the `ModalRoute.of()` method. This method returns the current route with the arguments.

```

class ExtractArgumentsScreen extends StatelessWidget {
  //const ExtractArgumentsScreen({Key? key}) : super(key: key);

  static const routeName = '/extractArguments';

  @override
  Widget build(BuildContext context) {
    // Extract the arguments from the current ModalRoute
    // settings and cast them as ScreenArguments.
    final args = ModalRoute.of(context)!.settings.arguments as
      ScreenArguments;

    return Scaffold(
      appBar: AppBar(
        title: Text(args.title),
      ),
      body: Center(
        child: Text(args.message),
      ),
    );
  }
}

```

3. Register the widget in the routes table

Next, add an entry to the `routes` provided to the `MaterialApp` widget. The `routes` define which widget should be created based on the name of the route.

```

MaterialApp(
  routes: {
    ExtractArgumentsScreen.routeName: (context) => const ExtractArgumentsScreen(),
  },
)

```

4. Navigate to the widget

Finally, navigate to the `ExtractArgumentsScreen` when a user taps a button using `Navigator.pushNamed()`. Provide the arguments to the route via the `arguments` property. The `ExtractArgumentsScreen` extracts the `title` and `message` from these arguments.

```

ElevatedButton( onPressed: () {

```

```

        Navigator.pushNamed(context,
            ExtractArgumentsScreen.routeName, arguments:
ScreenArguments('Ruqaih Salman',
    28, ), ));},
    child: const Text('Navigate to screen that extracts arguments'),
),

```

Interactive example

```

import 'package:flutter/material.dart';

void main() => runApp(Main_Page_Pass_Data_Roq());

class Main_Page_Pass_Data_Roq extends StatelessWidget {
  //const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData(
        primarySwatch: Colors.teal,
      ),

      debugShowCheckedModeBanner: false,
      routes: {
        ExtractArgumentsScreen_by_ModalRoute.routeName: (context) =>
          ExtractArgumentsScreen_by_ModalRoute(),
      },
      // Provide a function to handle named routes.
      // Use this function to identify the named
      // route being pushed, and create the correct
      // Screen.

      title: 'Navigation with Arguments Roq',
      home: HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  //const HomeScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Home Screen Roq'),
      ),
      body: Center(
        child: Container(
          padding: const EdgeInsets.all(15.0),
          alignment: Alignment.topCenter,
          child: ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: Theme.of(context).primaryColorLight,
            ),
            onPressed: () {
              // When the user taps the button,
              // navigate to a named route and
              // provide the arguments as an optional

```

```

        // parameter.
        Navigator.pushNamed(
            context,
            ExtractArgumentsScreen_by_ModalRoute.routeName,
            arguments: ScreenArguments(
                'Ruqaih Salman',
                28,
            ),
        );
    },
    child: const Text(
        'Navigate to screen that extracts arguments',
        style:
            TextStyle(color: Colors.black, fontWeight: FontWeight.bold),
    ),
),
),
);
}
}

// A Widget that extracts the necessary arguments from
// the ModalRoute.
class ExtractArgumentsScreen_by_ModalRoute extends StatelessWidget {
    //const ExtractArgumentsScreen({Key? key}) : super(key: key);

    static const routeName = '/extractArguments_by_ModalRoute';

    @override
    Widget build(BuildContext context) {
        // Extract the arguments from the current ModalRoute
        // settings and cast them as ScreenArguments.
        // -----Code of New Flutter SDK-----
        final ScreenArguments args =
            ModalRoute.of(context)!.settings.arguments as ScreenArguments;

        return Scaffold(
            appBar: AppBar(
                title: Text("User : ${args.name}.substring(0, 6)}"),
            ),
            body: Padding(
                padding: const EdgeInsets.all(15.0),
                child: Container(
                    alignment: Alignment.center,
                    child: Column(
                        children: [
                            Text(
                                "Your Name : ${args.name}",
                                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                            ),
                            Text(
                                "Your Age : ${args.age}",
                                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                            ),
                        ],
                    ),
                ),
            ),
        );
    }
}

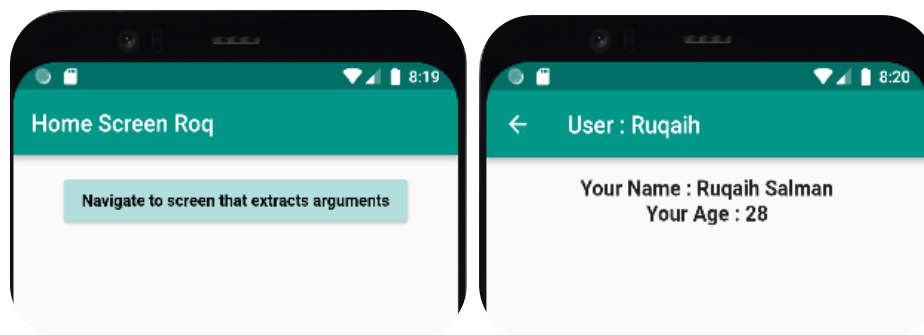
```

```
// A Widget that accepts the necessary arguments via the
// constructor.
// You can pass any object to the arguments parameter.
// In this example, create a class that contains both
// a customizable title and message.
class ScreenArguments {
  final String name;
  final int age;

  ScreenArguments(this.name, this.age);
}
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



Return data from a screen

In some cases, you might want to return data from a new screen. For example, say you push a new screen that presents two options to a user. When the user taps an option, you want to inform the first screen of the user's selection so that it can act on that information.

You can do this with the `Navigator.pop()` method using the following steps:

1. Define the home screen
2. Add a button that launches the selection screen
3. Show the selection screen with two buttons
4. When a button is tapped, close the selection screen
5. Show a snackbar on the home screen with the selection

1. Define the home screen

The home screen displays a button. When tapped, it launches the selection screen.

```
import 'package:flutter/material.dart';
class HomeScreen extends StatelessWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```

    appBar: AppBar(
      title: const Text('Returning Data Demo'),
    ),
    // Create the SelectionButton widget in the next step.
    body: const Center(
      child: SelectionButton(),
    ),
  );
}

class SelectionButton extends StatelessWidget {
  const SelectionButton({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        _navigateAndDisplaySelection(context);
      },
      child: const Text('Pick an option, any option!'),
    );
  }

  void _navigateAndDisplaySelection(BuildContext context) async {
    // Navigator.push returns a Future that completes after calling
    // Navigator.pop on the Selection Screen.
    final result = await Navigator.push(
      context,
      // Create the SelectionScreen in the next step.
      MaterialPageRoute(builder: (context) => const SelectionScreen()),
    );
  }
}

```

2. Show the selection screen with two buttons

Now, build a selection screen that contains two buttons. When a user taps a button, that app closes the selection screen and lets the home screen know which button was tapped. This step defines the UI. The next step adds code to return data.

```

class SelectionScreen extends StatelessWidget {
  const SelectionScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Pick an option'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Padding(
              padding: const EdgeInsets.all(8.0),
              child: ElevatedButton(
                onPressed: () {
                  // Pop here with "Yep"...
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        child: const Text('Yep!'),
      ),
    ),
    Padding(
      padding: const EdgeInsets.all(8.0),
      child: ElevatedButton(
        onPressed: () {
          // Pop here with "Nope"...
        },
        child: const Text('Nope.'),
      ),
    ),
  ],
),
),
);
}
}

```

3. When a button is tapped, close the selection screen

Now, update the `onPressed()` callback for both of the buttons. To return data to the first screen, use the `Navigator.pop()` method, which accepts an optional second argument called `result`. Any result is returned to the `Future` in the `SelectionButton`.

➤ Yep button

```

ElevatedButton(
  onPressed: () {
    // Close the screen and return "Yep!" as the result.
    Navigator.pop(context, 'Yep!');
  },
  child: const Text('Yep!'),
)

```

➤ Nope button

```

ElevatedButton(
  onPressed: () {
    // Close the screen and return "Nope." as the result.
    Navigator.pop(context, 'Nope.');
```

4. Show a snackbar on the home screen with the selection

Now that you're launching a selection screen and awaiting the result, you'll want to do something with the information that's returned. In this case, show a snackbar displaying the result by using the `_navigateAndDisplaySelection()` method in `SelectionButton`:

```

void _navigateAndDisplaySelection(BuildContext context) async {
  // Navigator.push returns a Future that completes after calling
  // Navigator.pop on the Selection Screen.
  final result = await Navigator.push(

```

```

    context,
    MaterialPageRoute(builder: (context) => const SelectionScreen()),
  );

  ScaffoldMessenger.of(context)
    ..removeCurrentSnackBar()
    ..showSnackBar(SnackBar(content: Text('$result')));
}

```

➤ Example: Complete Code

```

import 'package:flutter/material.dart';

void main() => runApp(HomeScreen_ReturnEx_Roq());

class HomeScreen_ReturnEx_Roq extends StatelessWidget {
  //--> const HomeScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData(
        primarySwatch: Colors.teal,
      ),
      title: 'Returning Data Roq App',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Returning Data Roq App'),
        ),
        body: Container(
          alignment: Alignment.topCenter,
          padding: EdgeInsets.only(top:20),

          child: SelectionButton(),
        ),
      ));
  }
}

class SelectionButton extends StatelessWidget {
  // --> const SelectionButton({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        _navigateAndDisplaySelection(context);
      },
      child: const Text('Pick an option, any option!'),
    );
  }

  // A method that launches the SelectionScreen and awaits the result from
  // Navigator.pop.
  void _navigateAndDisplaySelection(BuildContext context) async {
    // Navigator.push returns a Future that completes after calling
    // Navigator.pop on the Selection Screen.
    final result = await Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => //-->const
          SelectionScreen(),
      ),
    );
  }
}

```

```

);

// After the Selection Screen returns a result, hide any previous snackbars
// and show the new result.
ScaffoldMessenger.of(context)
  ..removeCurrentSnackBar()
  ..showSnackBar(SnackBar(content: Text('$result')));
}
}

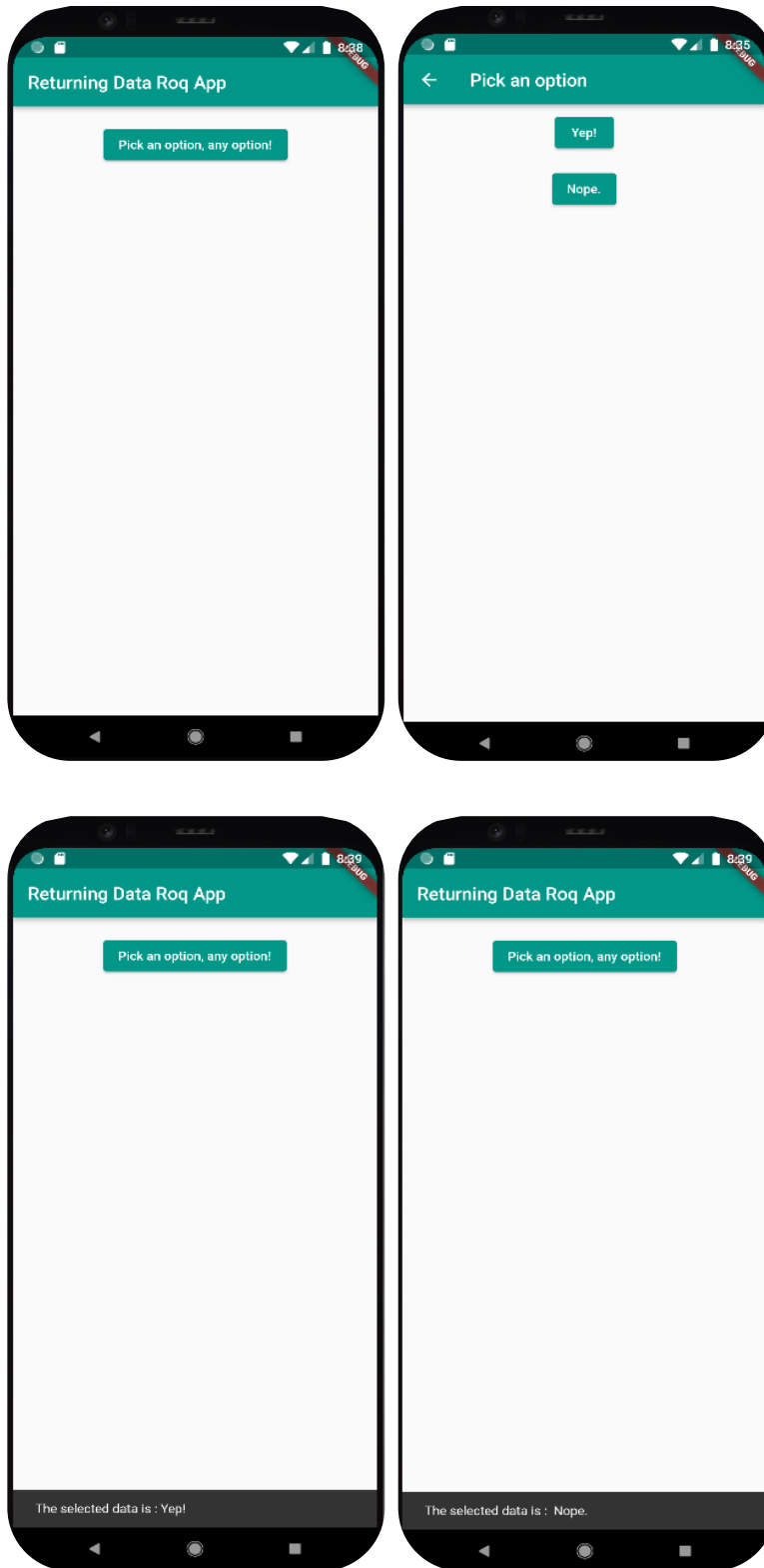
class SelectionScreen extends StatelessWidget {
  const SelectionScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Pick an option'),
      ),
      body: Center(
        child: Column(
          //mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Padding(
              padding: const EdgeInsets.all(8.0),
              child: ElevatedButton(
                onPressed: () {
                  // Close the screen and return "Yep!" as the result.
                  Navigator.pop(context, 'The selected data is : Yep!');
                },
                child: const Text('Yep!'),
              ),
            ),
            Padding(
              padding: const EdgeInsets.all(8.0),
              child: ElevatedButton(
                onPressed: () {
                  // Close the screen and return "Nope." as the result.
                  Navigator.pop(context, 'The selected data is : Nope. ');
                },
                child: const Text('Nope. '),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



Navigate Between Screens Using Hero Animation

The Hero widget is a great out-of-the-box animation to convey the navigation action of a widget flying into place from one page to another. The hero animation is a shared element transition (animation) between two different pages. To visualize the animation, imagine seeing a superhero flying into action. For example, you have a list of journal entries with a photo thumbnail, the user selects an entry, and you see the photo

thumbnail transition to the detail page by moving and growing to full size. The photo thumbnail is the superhero, and when tapped, it flies into action by moving from the list page to the detail page and lands perfectly on the correct location at the top of the detail page showing the full photo. When the detail page is dismissed, the Hero widget flies back to the original page, position, and size. In other words, the animation shows the photo thumbnail moving and growing into place from the list page to the detail page, and once the detail page is dismissed, the animation and size are reversed. The Hero widget has all of these features built in; there's no need to write custom code to handle the size and animation between pages. To continue with the previous scenario, you wrap the list page Image widget as a child of the Hero widget and assign a tag property name. Repeat the same steps for the detail page and make sure the tag property value is the same in both pages.

```
// List page
Hero(
  tag: 'photo1',
  child: Image(
    image: AssetImage("assets/images/coffee.png"),
  ),
),
// Detail page
Hero(
  tag: 'photo1',
  child: Container(
    child: Image(
      image: AssetImage("assets/images/coffee.png"),
    ),
  ),
),
```

The Hero child widget is marked for hero animation. When the Navigator pushes or pops a PageRoute, the entire screen's content is replaced. This means during the animation transition the Hero widget is not shown in the original position in both the old and new routes, but it moves and resizes from one page to another. Each Hero tag must be unique and match on both the originating and landing pages.

Animate a widget across screens

It's often helpful to guide users through an app as they navigate from screen to screen. A common technique to lead users through an app is to animate a widget from one screen to the next. This creates a visual anchor connecting the two screens.

Use the **Hero** widget to animate a widget from one screen to the next. This recipe uses the following steps:

- Create two screens showing the same image.
- Add a Hero widget to the first screen.
- Add a Hero widget to the second screen.

1. Create two screens showing the same image

In this example, display the same image on both screens. Animate the image from the first screen to the second screen when the user taps the image. For now, create the visual structure; handle animations in the next steps.

```
import 'package:flutter/material.dart';

class MainScreen extends StatelessWidget {
  const MainScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Main Screen'),
      ),
      body: GestureDetector(
        onTap: () {
          Navigator.push(context, MaterialPageRoute(builder: (context) {
            return const DetailScreen();
          }));
        },
        child: Image.network(
          'https://picsum.photos/250?image=9',
        ),
      ),
    );
  }
}

class DetailScreen extends StatelessWidget {
  const DetailScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: GestureDetector(
        onTap: () {
          Navigator.pop(context);
        },
        child: Center(
          child: Image.network(
            'https://picsum.photos/250?image=9',
          ),
        ),
      ),
    );
  }
}
```

2. Add a Hero widget to the first screen

To connect the two screens together with an animation, wrap the `Image` widget on both screens in a `Hero` widget. The `Hero` widget requires two arguments:

- ``tag`` : An object that identifies the ``Hero``. It must be the same on both screens.

- **`child`** : The widget to animate across screens.

```
Hero(  
  tag: 'imageHero',  
  child: Image.network(  
    'https://picsum.photos/250?image=9',  
  ),  
)
```

3. Add a Hero widget to the second screen

To complete the connection with the first screen, wrap the `Image` on the second screen with a `Hero` widget that has the same `tag` as the `Hero` in the first screen.

After applying the `Hero` widget to the second screen, the animation between screens just works.

```
Hero(  
  tag: 'imageHero',  
  child: Image.network(  
    'https://picsum.photos/250?image=9',  
  ),  
)
```

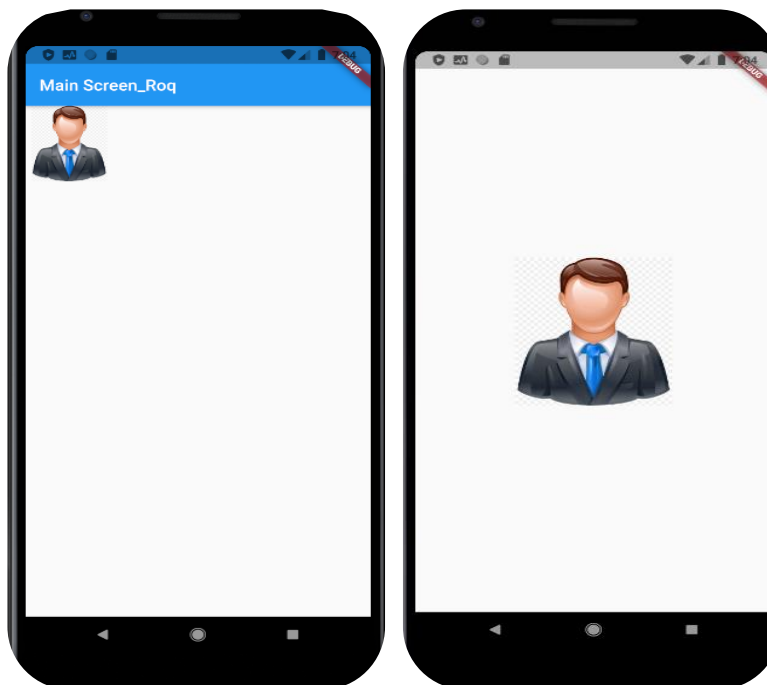
Completed Code example

```
import 'package:flutter/material.dart';  
  
void main() => runApp(Animate_a_widget_across_screens_Roq_Ex());  
  
class Animate_a_widget_across_screens_Roq_Ex extends StatelessWidget {  
  //-->const HeroApp({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Navigatio with Animation Roq',  
      home: MainScreen_Roq(),  
    );  
  }  
}  
  
class MainScreen_Roq extends StatelessWidget {  
  //-->const MainScreen({Key? key}) : super(key: key);  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('Main Screen_Roq'),  
      ),  
      body: GestureDetector(  
        onTap: () {  
          Navigator.push(context, MaterialPageRoute(builder: (context) {  
            return DetailScreen_Roq();  
          }));  
        },  
        child: Hero(  
          tag: 'imageHero_roq',  
          //child: Image.network('https://picsum.photos/250?image=9',  
          child: Image.asset('Image_roq/roq_2.jpg',width: 100.0, height: 100.0),  
        ), ), );  
  }  
}  
  
class DetailScreen_Roq extends StatelessWidget {  
  //-->const DetailScreen({Key? key}) : super(key: key);
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: GestureDetector(
      onTap: () {
        Navigator.pop(context);
      },
      child: Center(
        child: Hero(
          tag: 'imageHero_roq',
          //child: Image.network('https://picsum.photos/250?image=9'),
          child: Image.asset('Image_roq/roq_2.jpg', width: 200.0, height: 200.0),
        ),
      ),
    ),
  );
}
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



Navigation with Tabbar

In this section, we are going to learn how the tab bar works in Flutter. The tabs are mainly used for mobile navigation. The styling of tabs is different for different operating systems. For example, it is placed at the top of the screen in android devices while it is placed at the bottom in iOS devices.

Working with tabs is a common pattern in Android and iOS apps that follow the Material Design guidelines. Flutter provides a convenient way to create a tab layout. To add tabs to the app, we need to create a **TabBar** and **TabBarView** and attach them with the TabController. The controller will sync both so that we can have the behavior which we need.

Let us see step by step to create a tab bar in Flutter application.

- **Step 1:** First, you need to create a Flutter project in your IDE. Here, I am going to use Android Studio.
- **Step 2:** Open the app in Android Studio and navigate to the lib folder. Inside the lib folder, create two dart files and named it **FirstScreen** and **SecondScreen**.

Write the following code in the FirstScreen_Roq:

```
import 'package:flutter/material.dart';

class FirstScreen_Roq extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Container(
      child: Center(
        child: Text('It is a contact tab, which is responsible for displaying the
                     contacts stored in your mobile',
                     style: TextStyle(fontSize: 32.0),
                  ),
      ),
    );
  }
}
```

Write the following code in the SecondScreen_Roq

```
import 'package:flutter/material.dart';

class SecondScreen_Roq extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Container(
      child: Center(
        child: Text('It is a second layout tab, which is responsible for taking pictures
                     from your mobile.',
                     style: TextStyle(fontSize: 35.0),
                  ),
      ),
    );
  }
}
```

- **Step 3:** Next, we need to create a **DefaultTabController**. The **DefaultTabController** creates a **TabController** and makes it available to all widgets.

```
DefaultTabController(
  // The number of tabs to display.
  length: 2,
  child: // Complete this code in the next step.
);
```

In the above code, the **length** property tells about the number of tabs used in the app.

- **Step 4:** Create the tab. We can create tabs by using the **TabBar** widget as below code.

```
DefaultTabController(
  length: 2,
```

```

child: Scaffold(
  appBar: AppBar(
    bottom: TabBar(
      tabs: [
        Tab(icon: Icon(Icons.directions_car)),
        Tab(icon: Icon(Icons.directions_bike)),
      ],
    ),
  ),
),
);

```

- **Step 5:** Create content for each tab so that when a tab is selected, it displays the content. For this purpose, we have to use the **TabBarView** widget as:

```

TabBarView(
  children: [
  ],
);

```

- **Step 6:** Finally, open the main.dart file and insert the following code

```

import 'package:flutter/material.dart';

import 'FirstScreen_Roq.dart';
import 'SecondScreen_Roq.dart';

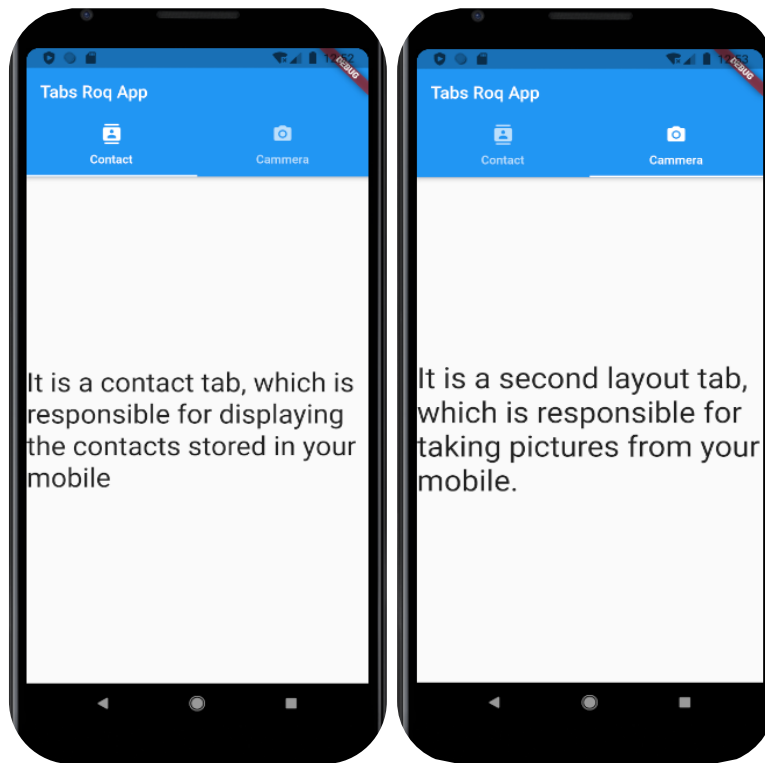
void main() => runApp(TebBar_Roq_Ex());

class TebBar_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: DefaultTabController(
        length: 2,
        child: Scaffold(
          appBar: AppBar(
            title: Text('Tabs Roq App'),
            bottom: TabBar(
              tabs: [
                Tab(icon: Icon(Icons.contacts), text: "Contact"),
                Tab(icon: Icon(Icons.camera_alt), text: "Cammera")
              ],
            ),
          ),
          body: TabBarView(
            children: [
              FirstScreen_Roq(),
              SecondScreen_Roq(),
            ],
          ),
        ),
      ),
    );
  }
}

```

Output

Now, run the app in your Android Studio. It will give the following screen where you can see two tab icons. So when you click any of the tab icons, it will display the associated screen.



Navigation with Dialog and Picker:

The dialog is a type of widget which comes on the window or the screen which contains any critical information or can ask for any decision. When a dialog box is popped up the all the other functions get disabled until you close the dialog box or provide an answer. We use a dialog box for a different type of condition such as alert notification, simple notification in which different options are shown, or we can also make a dialog box which can be used as a tab for showing the dialog box.

Types of dialogs in a flutter:

1. AlertDialog
2. Simple Dialog
3. TimePicker Dialog
4. DatePicker Dialog
5. Image Picker Dialog

AlertDialog

AlertDialog is a material widget in flutter which is used to take confirmation before performing an action. For example, when we try to log out of an application it will ask confirmation with yes and no actions. We have to use alert dialog in such cases. An alert dialog has three important properties title, content and actions which are optional. Even though the content is optional there is no point in using an alert dialog

without content. The title is displayed at the top & content will be displayed below the title. The actions will be displayed at the bottom of the dialog.

Constructor:

```
AlertDialog(  
  {  
    Key key,  
    Widget title,  
    EdgeInsetsGeometry titlePadding,  
    TextStyle titleTextStyle,  
    Widget content,  
    EdgeInsetsGeometry contentPadding: const EdgeInsets.fromLTRB(24.0, 20.0, 24.0, 24.0),  
    TextStyle contentTextStyle,  
    List<Widget> actions,  
    EdgeInsetsGeometry actionsPadding: EdgeInsets.zero,  
    VerticalDirection actionsOverflowDirection,  
    double actionsOverflowButtonSpacing,  
    EdgeInsetsGeometry buttonPadding,  
    Color backgroundColor,  
    double elevation,  
    String semanticLabel,  
    EdgeInsets insetPadding: _defaultInsetPadding,  
    Clip clipBehavior: Clip.none,  
    ShapeBorder shape,  
    bool scrollable: false  
  }  
)
```

Creating AlertDialog In Flutter

We can create an **AlertDialog** in flutter by using the constructor. To display the **alert dialog** we have to use **showDialog()** method. We will return alertDialog as the child widget to showDialog, which displays the dialog. Let's create an alert dialog and display it. First, we have to design the UI and a function where we will call the showDialog method that displays the alert dialog. Let's get to work.

Designing the UI

We will design the UI where we will display a Button widget and a text widget. Clicking on the button will call the showDialog method that will show the AlertDialog. When the user chooses an option we will display the selected option in the text widget.

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: Text("Flutter AlertDialog"),  
    ),  
    body: Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: <Widget>[  
          ElevatedButton(  
            onPressed: () {  
              _displayDialog(context);  
            }  
          )  
        ],  
      ),  
    ),  
  );  
}
```

```

        },
        child: Text("Show Dialog"),
      ),
      Text("${selectedTime.hour}:${selectedTime.minute}"),
    ],
  ),
),
);
}

```

Designing UI Of AlertDialog

Let's design the UI for alertDialog where we will have a title, message, and two actions YES and NO. The title and message are text widgets and the actions are button widgets.

```

AlertDialog(
  title: Text('Welcome'),
  content: Text('Do you wanna learn flutter?'),
  actions: [
    TextButton(
      onPressed: () {
        Navigator.of(context).pop();
      },
      child: Text('YES', style: TextStyle(color: Colors.black)),
    ),
    TextButton(
      onPressed: () {
        Navigator.of(context).pop();
      },
      child: Text('NO', style: TextStyle(color: Colors.black)),
    ),
  ],
),

```

Creating the showDialog() method

Now we will create a function **_displayDialog** where we call the **showDialog()** method which will display AlertDialog with title, message, and two buttons. If you don't want the user to dismiss the dialog without selecting an action, set **barrierDismissible** property of showDialog method to **false**.

```

_displayDialog(BuildContext context) async {
  showDialog(
    barrierDismissible:false,
    context: context,
    builder: (BuildContext context) {
      return Expanded(
        child: AlertDialog(
          title: Text('Welcome'),
          content: Text('Do you wanna learn flutter?'),
          actions: [
            TextButton(
              onPressed: () {
                Navigator.of(context).pop();
              },
              child: Text('YES', style: TextStyle(color: Colors.black)),
            ),
            TextButton(
              onPressed: () {

```

```

        Navigator.of(context).pop();
      },
      child: Text('NO', style: TextStyle(color: Colors.black)),
    ),
  ],
),
);
},
);
}
}

```

➤ Example: AlertDialog

Let's see an example where we create and display an alert dialog in flutter. For creating dialog will use AlertDialog widget and add two text buttons as actions. To display the dialog we will use showDialog function.

```

import 'package:flutter/material.dart';
void main()
{
  runApp(AleartDialog_Roq_EX());
}
class AleartDialog_Roq_EX extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'AlertDialog Roq App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  @override
  _MyHomePageState createState()
  {
    return _MyHomePageState();
  }
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("AlertDialog Roq App"),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            ElevatedButton(
              onPressed: () {
                _showDialog_roq_fun(context);
              },
              child: Text("Show Dialog "),
            ),
          ],
        ),
      ),
    );
  }
}

```

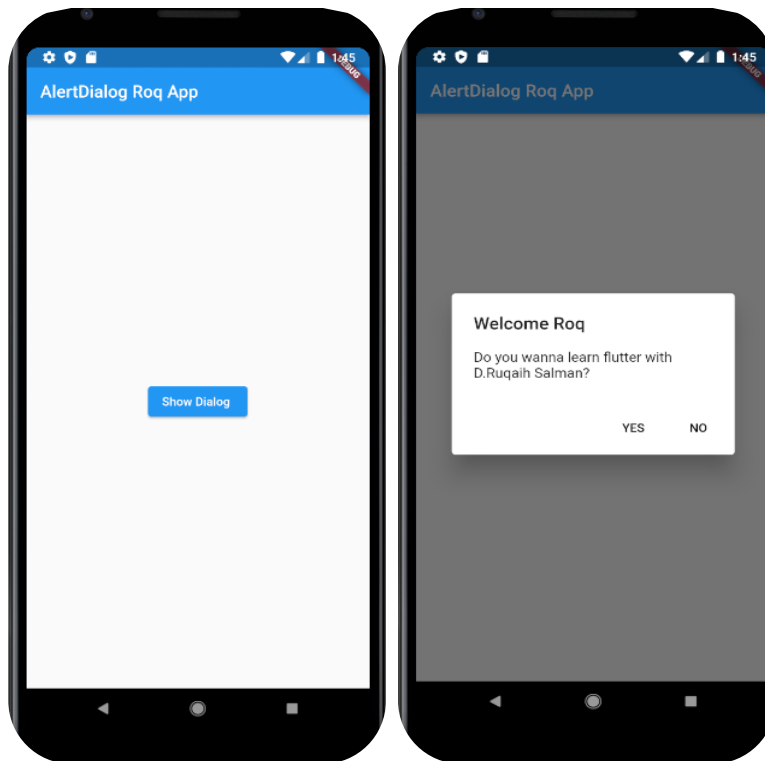
```

    ),
    ],
  ),
),
);
}
_showDialog_roq_fun(BuildContext context) {
  showDialog(
    barrierDismissible:false,
    context: context,
    builder: (BuildContext context) {
      return Expanded(
        child: AlertDialog(
          title: Text('Welcome Roq'),
          content: Text('Do you wanna learn flutter with D.Ruqaih Salman?'),
          actions: [
            TextButton(
              onPressed: () {
                Navigator.of(context).pop();
              },
              child: Text('YES', style: TextStyle(color: Colors.black),),
            ),
            TextButton(
              onPressed: () {
                Navigator.of(context).pop();
              },
              child: Text('NO', style: TextStyle(color: Colors.black),),
            ),
          ],
        ),
      ),
    ),
  );
},
);
}
}

```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



Properties of AlertDialog

The most used properties of AlertDialog are:

- title
- content
- actions
- backgroundColor
- elevation

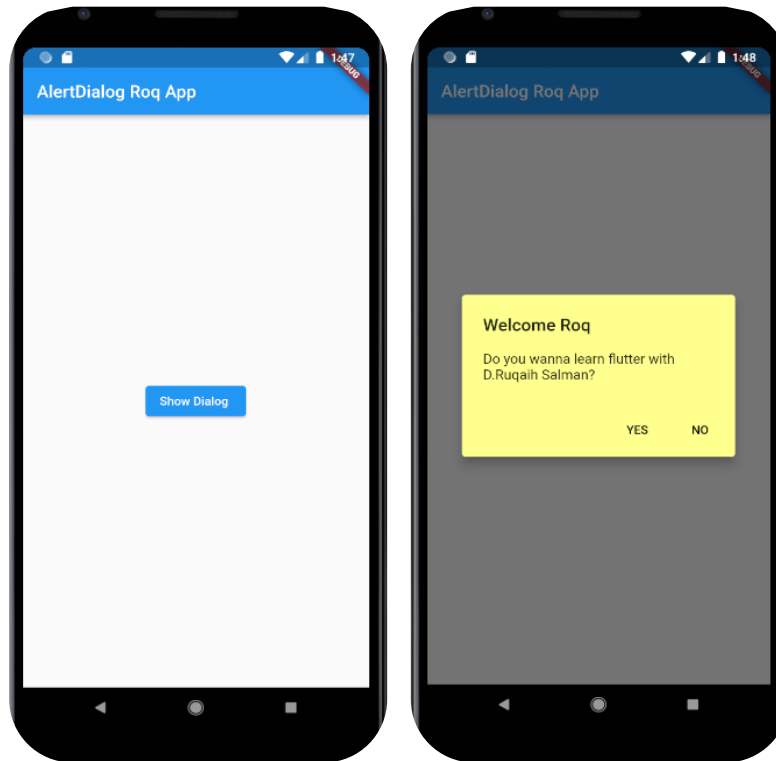
Since we have already seen the title, content actions in the above code example let's check out background and elevation. How to change background color and elevation of **Alertdialog**

```
AlertDialog(
  title: Text('Welcome Roq'),
  content: Text('Do you wanna learn flutter with D.Ruqaih Salman?'),
  actions: [
    TextButton(
      onPressed: () {
        Navigator.of(context).pop();
      },
      child: Text('YES', style: TextStyle(color: Colors.black)),
    ),
    TextButton(
      onPressed: () {
        Navigator.of(context).pop();
      },
      child: Text('NO', style: TextStyle(color: Colors.black)),
    ),
  ],
  elevation: 10,
```

```
backgroundColor: Colors.yellowAccent.shade100,  
) ,
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



SimpleDialog

SimpleDialog is a material widget in flutter that lets the user choose an option from a set of options. A simple dialog has a title property that is optional. We can use simple dialogs in situations where the user has to pick a single item from a list of items. For example, while making payments we will be displayed with a list of payment options where we have to choose one payment option. We will create the options using the SimpleDialogOption widget. This tutorial is about how to use **SimpleDialog** widget in **flutter**. We will also customize its style with different properties and see an example. If you just want to display a message to the user instead of providing options consider using [AlertDialog](#).

Constructor:

```
SimpleDialog({  
  Key key,  
  Widget title,  
  EdgeInsetsGeometry titlePadding,  
  TextStyle titleTextStyle,  
  List<Widget> children,  
  EdgeInsetsGeometry contentPadding,  
  Color backgroundColor,  
  double elevation,
```

```
String semanticLabel,
EdgeInsets insetPadding,
Clip clipBehavior,
ShapeBorder shape
})
```

Creating SimpleDialog In Flutter

We can create a `SimpleDialog` in flutter by using its constructor. To display the **simple dialog** we have to use `showDialog()` method. We will return `SimpleDialog` as the child widget to `showDialog`, which displays the dialog. Let's create a simple dialog and display it. First, we have to design the UI and a function where we will call the `showDialog` method that displays the alert dialog.

Designing The UI

We will design the UI where we will display a `Button` widget and a `Text` widget. Clicking on the button will call the `showDialog` method that will show the `Simple dialog`. When the user chooses an option we will display the selected option in the `Text` widget.

```
var _selected = "";
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("Flutter SimpleDialog"),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          ElevatedButton(
            onPressed: () {
              _displayDialog(context);
            },
            child: Text("Show Dialog"),
          ),
          Text(_selected)
        ],
      ),
    ),
  );
}
```

Designing UI Of SimpleDialog

We will now design a simple dialog where we will have a title and two options. The title is a `Text` widget and the options are `SimpleDialogOption` widgets.

```
SimpleDialog(
  title: Text('Choose food'),
  children: [
    SimpleDialogOption(
      onPressed: () {
        Navigator.pop(context, "Pizza");
      },
      child: const Text('Pizza'),
    ),
  ],
)
```

```

        ),
        SimpleDialogOption(
          onPressed: () {
            Navigator.pop(context, "Burger");
          },
          child: const Text('Burger'),
        ),
      ],
      elevation: 10,
      //backgroundColor: Colors.green,
    ),
  ),

```

Creating the showDialog() method

Now let's create a method `_displayDialog()` where we call the `showDialog()` function which will display `SimpleDialog` widget with a title, and two options. when the user chooses an option we are assigning it to a variable (`_selected`) and updating the text view inside the `setState()`.

```

_displayDialog(BuildContext context) async {
  _selected = await showDialog(
    context: context,
    builder: (BuildContext context) {
      return Expanded(
        child: SimpleDialog(
          title: Text('Choose food'),
          children: [
            SimpleDialogOption(
              onPressed: () {
                Navigator.pop(context, "Pizza"); },
              child: const Text('Pizza'),
            ),
            SimpleDialogOption(
              onPressed: () {
                Navigator.pop(context, "Burger");
              },
              child: const Text('Burger'),
            ),
          ],
          elevation: 10,
          //backgroundColor: Colors.green,
        ),
      );
    },
  );
  setState(() {
    _selected = _selected;
  });
}

```

If you don't want the user to dismiss the dialog without selecting an option, set `barrierDismissible` property of `showDialog` method to `false`.

```

showDialog(
  context: context,
  barrierDismissible: false,
  builder: (BuildContext context) {
    return Expanded(
      child: SimpleDialog(

```

```

        title: Text('Choose food'),
        children:[
          SimpleDialogOption(
            onPressed: () {
              Navigator.pop(context, "Pizza"); },
            child: const Text('Pizza'),
          ),
          SimpleDialogOption(
            onPressed: () {
              Navigator.pop(context, "Burger");
            },
            child: const Text('Burger'),
          ),
        ],
        elevation: 10,
        backgroundColor: Colors.yellowAccent,
        shape: StadiumBorder(),
      ),
    ),
  ],
);
}
);

```

➤ Example: Complete Code of SimpleDialog

Below is the complete code example for creating a simple dialog in flutter and displaying it.

```

import 'package:flutter/material.dart';
void main()
{
  runApp(SimpleDialog_Roq_EX());
}
class SimpleDialog_Roq_EX extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'SimpleDialog Roq App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  @override
  _MyHomePageState createState()
  {
    return _MyHomePageState();
  }
}

class _MyHomePageState extends State<MyHomePage> {
  var _selected = "";
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("SimpleDialog Roq App"),

```

```

    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          ElevatedButton(
            onPressed: () {
              _displayDialog(context);
            },
            child: Text("Show SimpleDialog"),
          ),
          Text(_selected)
        ],
      ),
    ),
  );
}

```

```

_displayDialog(BuildContext context) async {
  _selected = await showDialog(
    context: context,
    builder: (BuildContext context) {
      return Expanded(
        child: SimpleDialog(
          title: Text('Choose food fro Menu Roq'),
          children: [
            SimpleDialogOption(
              onPressed: () {
                Navigator.pop(context, "Pizza"); },
              child: const Text('Pizza'),
            ),
            SimpleDialogOption(
              onPressed: () {
                Navigator.pop(context, "Shorma"); },
              child: const Text('Shorma'),
            ),
            SimpleDialogOption(
              onPressed: () {
                Navigator.pop(context, "Chicken"); },
              child: const Text('Chicken'),
            ),
            SimpleDialogOption(
              onPressed: () {
                Navigator.pop(context, "Burger");
              },
              child: const Text('Burger'),
            ),
          ],
          elevation: 10,
          //backgroundColor: Colors.green,
        ),
      );
    },
  );

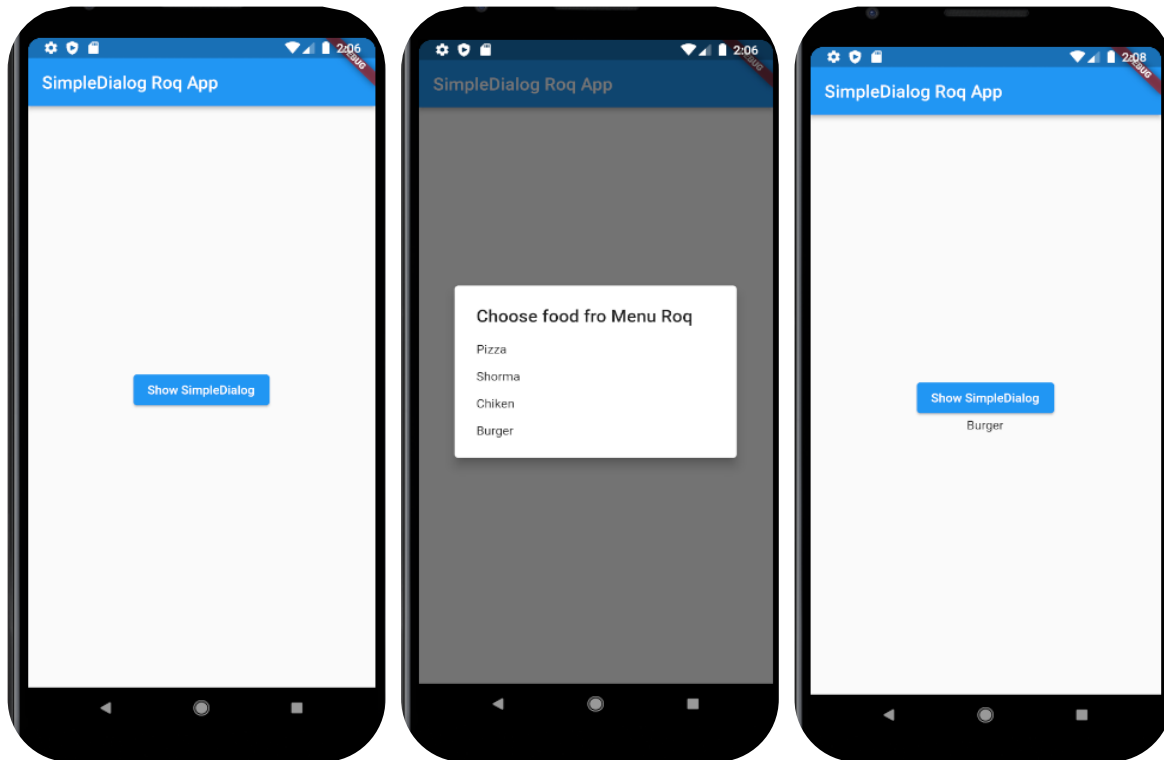
  if(_selected != null)
  {
    setState(() {
      _selected = _selected;
    });
  }
}

```

```
}  
}
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



How to change background color and elevation of SimpleDialog

```
SimpleDialog(  
  title: Text('Choose food fro Menu Roq'),  
  children: [  
    SimpleDialogOption(  
      onPressed: () {  
        Navigator.pop(context, "Pizza"); },  
      child: const Text('Pizza'),  
    ),  
    SimpleDialogOption(  
      onPressed: () {  
        Navigator.pop(context, "Shorma"); },  
      child: const Text('Shorma'),  
    ),  
    SimpleDialogOption(  
      onPressed: () {  
        Navigator.pop(context, "Chiken"); },  
      child: const Text('Chiken'),  
    ),  
    SimpleDialogOption(  
      onPressed: () {  
        Navigator.pop(context, "Burger");  
      },  
      child: const Text('Burger'),  
    ),  
  ],  
)
```

```

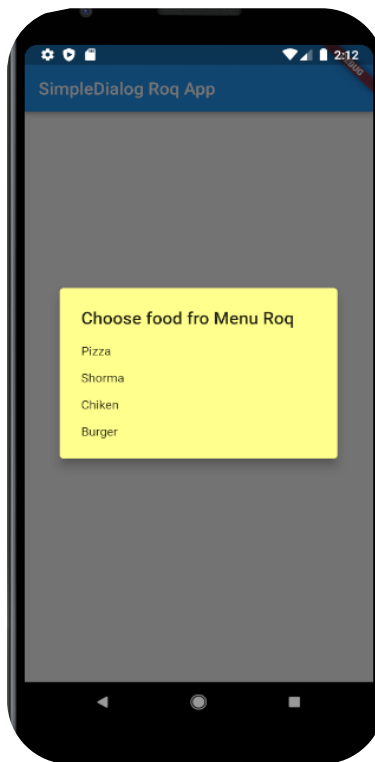
    ),
  ],
  elevation: 10,
  backgroundColor: Colors.yellowAccent.shade100,

  //backgroundColor: Colors.green,
),

```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



How to change shape of simplifiedialog

In the below example I've used stadiumborder for the shape of simplifiedialog.

```

SimpleDialog(
  title: Text('Choose food fro Menu Roq'),
  children: [
    SimpleDialogOption(
      onPressed: () {
        Navigator.pop(context, "Pizza"); },
      child: const Text('Pizza'),
    ),
    SimpleDialogOption(
      onPressed: () {
        Navigator.pop(context, "Shorma"); },
      child: const Text('Shorma'),
    ),
    SimpleDialogOption(
      onPressed: () {
        Navigator.pop(context, "Chiken"); },
      child: const Text('Chiken'),
    ),
  ],
)

```

```

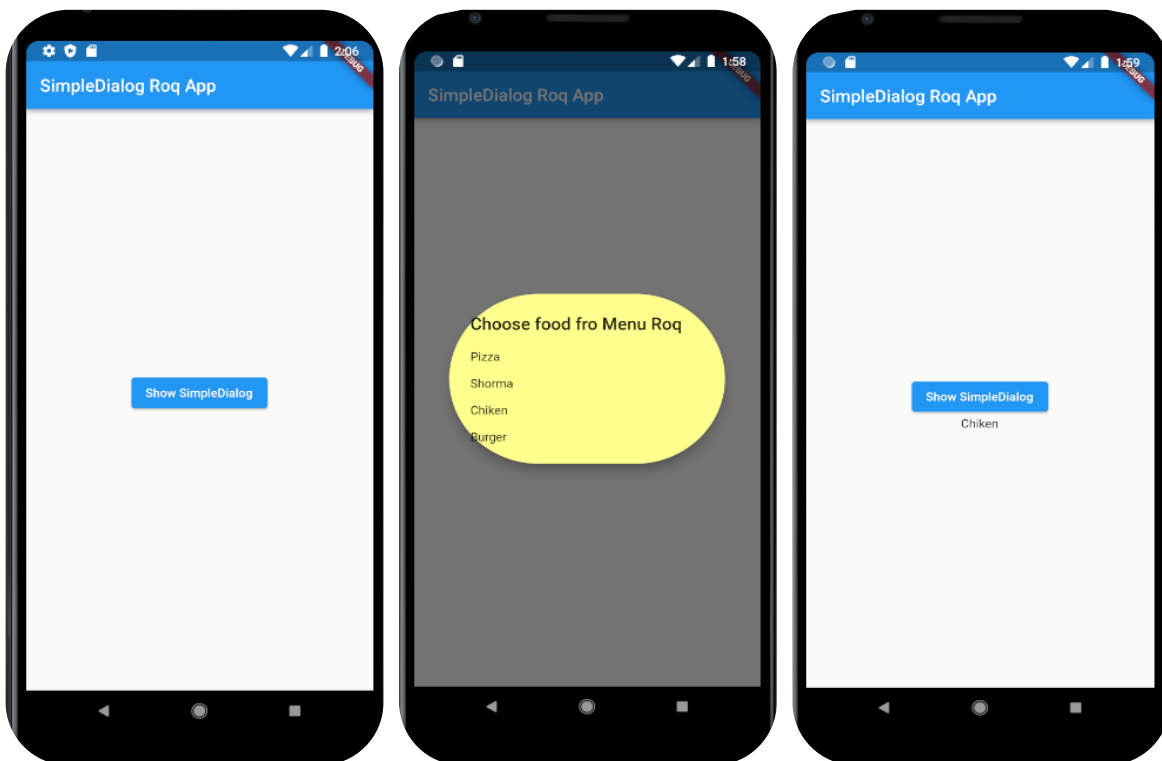
),
SimpleDialogOption(
  onPressed: () {
    Navigator.pop(context, "Burger");
  },
  child: const Text('Burger'),
),
],
elevation: 10,
backgroundColor: Colors.yellowAccent.shade100,
shape: StadiumBorder(),

//backgroundColor: Colors.green,
),

```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



TimePicker

TimePicker is a material widget in flutter that lets the user pick a time. Since there is no widget available for creating timepicker we will use `showTimePicker()` function. It will display a Material Design time picker in a dialog by calling flutter's inbuilt function. We will use it in instances like scheduling meetings, setting alarms, etc. To create a **TimePicker** we use **`showTimePicker()`** which is an asynchronous function. When we call the function it will display the dialog to the user to choose the time. It returns the selected time when the user confirms the dialog. If the user cancels the dialog, it will return null.

Constructor:

```
showTimePicker (  
{  
  required BuildContext context,  
  required TimeOfDay initialTime,  
  TransitionBuilder? builder,  
  bool useRootNavigator: true,  
  TimePickerEntryMode initialEntryMode: TimePickerEntryMode.dial,  
  String? cancelText,  
  String? confirmText,  
  String? helpText,  
  RouteSettings? routeSettings  
}  
)
```

TimePicker has one required property `initialTime` without which we cannot use the Time Picker. When the time picker is first displayed, it will show the **initialTime** selected by default.

Creating TimePicker

To create a **Timepicker** in flutter we have to initialize a variable **selectedTime** of type **TimeOfDay** which will hold the selected time. But initially, we will assign today's time to it.

```
TimeOfDay selectedTime = TimeOfDay.now();
```

Now let's design the UI where we will have an ElevatedButton widget to display the TimePicker dialog and a text widget to display the selected time.

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: Text("Flutter TimePicker"),  
    ),  
    body: Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: <Widget>[  
          ElevatedButton(  
            onPressed: () {  
              _selectTime(context);  
            },  
            child: Text("Choose Time"),  
          ),  
          Text("${selectedTime.hour}:${selectedTime.minute}"),  
        ],  
      ),  
    ),  
  );  
}
```

As the UI is ready now let's create the method to display TimePicker. We will be using the TimeOfDay variable '**selectedTime**' created above for **initialDate**. We are assigning the user selected time to the variable in **setState()** to update the UI.

```

_selectTime(BuildContext context) async {
  final TimeOfDay timeOfDay = await showTimePicker(
    context: context,
    initialTime: selectedTime,
    initialEntryMode: TimePickerEntryMode.dial,
  );
  if(timeOfDay != null && timeOfDay != selectedTime)
  {
    setState(() {
      selectedTime = timeOfDay;
    });
  }
}

```

➤ Example: Timepicker

Below is the complete code example where we create a timepicker and display it using the showTimePicker function provided by flutter.

```

import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
void main()
{
  runApp(Timepicker_Roq_Ex());
}
class Timepicker_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'TimePicker Roq App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  @override
  _MyHomePageState createState()
  {
    return _MyHomePageState();
  }
}

class _MyHomePageState extends State<MyHomePage> {
  TimeOfDay selectedTime_roq = TimeOfDay.now();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("TimePicker Roq App"),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            ElevatedButton(
              onPressed: () {

```

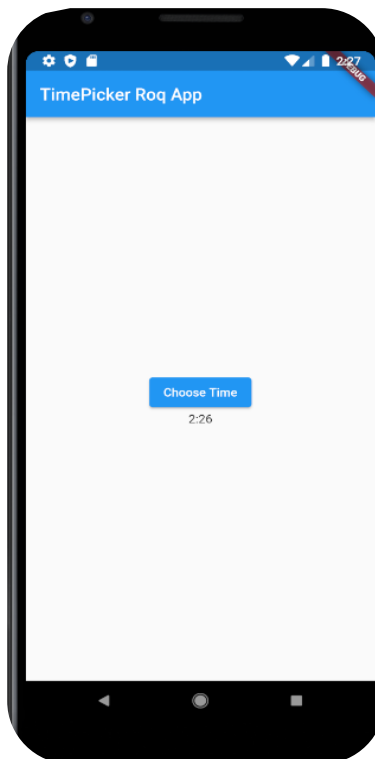
```

        _selectTime(context);
      },
      child: Text("Choose Time"),
    ),
    Text("${selectedTime_roq.hour}:${selectedTime_roq.minute}"),
  ],
),
),
);
}
_selectTime(BuildContext context) async {
  final TimeOfDay? timeOfDay = await showTimePicker(
    context: context,
    initialTime: selectedTime_roq,
    initialEntryMode: TimePickerEntryMode.dial,
  );
  if(timeOfDay != null && timeOfDay != selectedTime_roq)
  {
    setState(() {
      selectedTime_roq = timeOfDay;
    });
  }
}
}
}

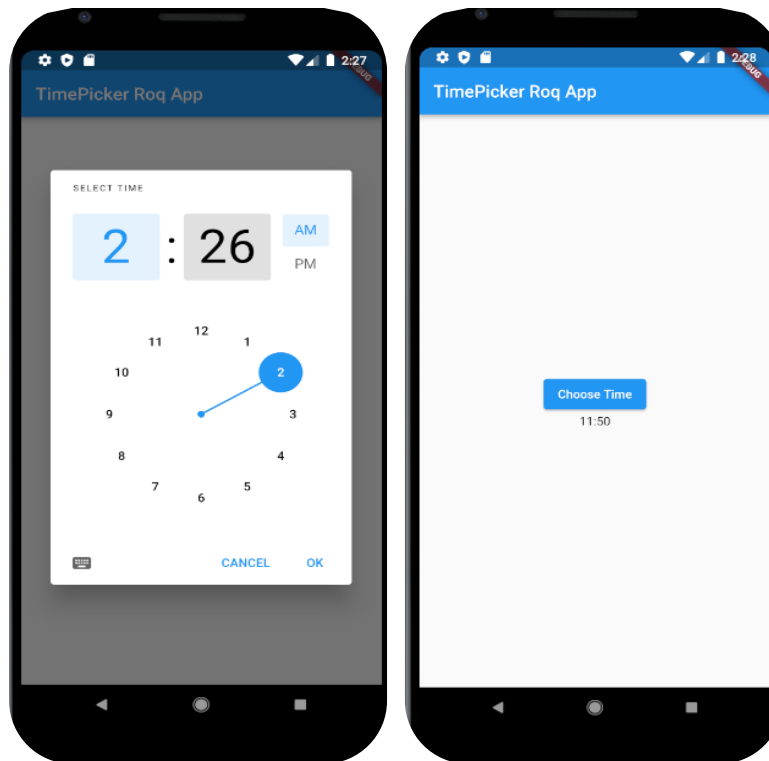
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



After clicking the button the time picker will be displayed.

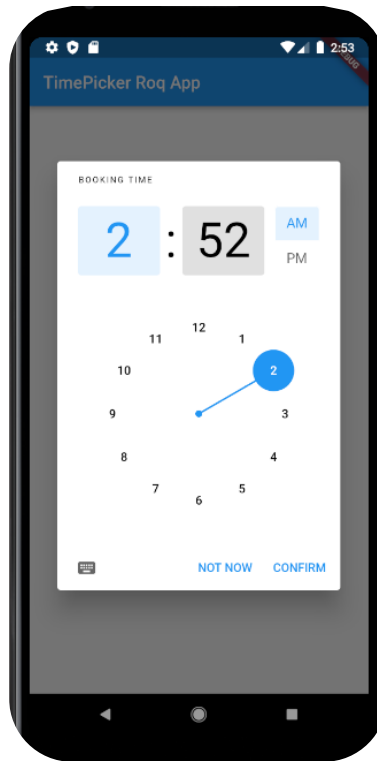


How to change help text, cancel and ok buttons text of timepicker

```
showTimePicker(
  context: context,
  initialTime: selectedTime_roq,
  initialEntryMode: TimePickerEntryMode.dial,
  //How to change help text, cancel and ok buttons text of timepicker
  confirmText: "CONFIRM",
  cancelText: "NOT NOW",
  helpText: "BOOKING TIME");
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



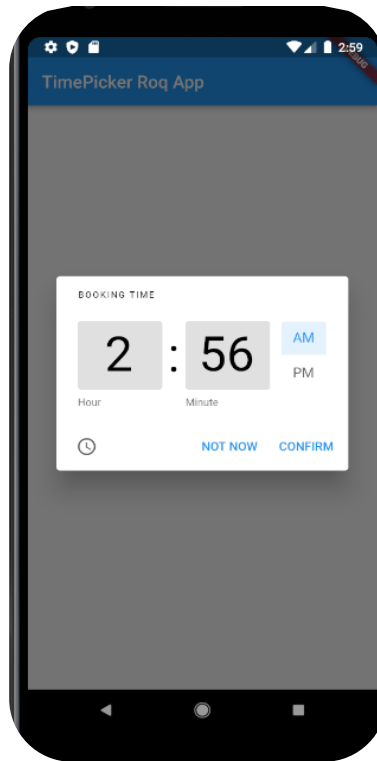
How to show a Text Input instead of dialer in timepicker

In flutter you can choose to display a text input instead of the dialer.

```
showTimePicker(  
  context: context,  
  initialTime: selectedTime_roq,  
  //How to show a Text Input instead of dialer in timepicker  
  initialEntryMode: TimePickerEntryMode.input,  
  confirmText: "CONFIRM",  
  cancelText: "NOT NOW",  
  helpText: "BOOKING TIME"),
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



DatePicker

DatePicker is a material widget in flutter that lets the user select a date. Since there is no widget available for creating datepicker we will use `showDatePicker()` function. It will display a Material Design date picker in a dialog by calling flutter's inbuilt function. We will use it in instances like booking a movie ticket, train ticket, etc. To create a **DatePicker** in Flutter we will use **`showDatePicker()`** which is an asynchronous function. When we call the function it will display the dialog where the user can select a date. It returns the selected date when the user confirms the dialog. If the user cancels the dialog, it will return null. We can display the date anywhere or save it in shared preferences for future use.

Constructor:

```
showDatePicker (
  {@required BuildContext context,
  @required DateTime initialDate,
  @required DateTime firstDate,
  @required DateTime lastDate,
  DateTime currentDate,
  DatePickerEntryMode initialEntryMode:
  DatePickerEntryMode.calendar,
  SelectableDayPredicate selectableDayPredicate,
  String helpText,
  String cancelText,
  String confirmText,
  Locale locale,
  bool useRootNavigator: true,
  RouteSettings routeSettings,
  TextDirection textDirection,
  TransitionBuilder builder,
  DatePickerMode initialDatePickerMode: DatePickerMode.day,
  String errorFormatText,
```

```
String errorInvalidText,
String fieldHintText,
String fieldLabelText
}
)
```

DatePicker has three required properties **initialDate**, **firstDate** and **lastDate** without which we cannot use the date Picker.

- **firstDate** : It is the earliest date we want the user to choose from.
- **lastDate** : It is the max possible date the user can select.
- **initialDate** : It is the date we want to display when the date picker is displayed for the first time. It should be between firstDate and lastDate.

The date picker will show the month of the date we have provided to the initialDate, with initialDate selected when it is displayed for the first time.

The currentDate represents today's date & it will be highlighted in the day grid. If we don't provide any value, it will use the date of DateTime.now().

Creating Datepicker

To create a datepicker in flutter we have to initialize a variable **selectedDate** of type **DateTime** which will hold the selected date. But initially we will assign today's date to it.

```
DateTime selectedDate = DateTime.now();
```

Now let's design the UI where we will have an ElevatedButton widget to display the DatePicker dialog. When the user selects the date we will display it in a text widget.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("Flutter Datepicker"),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          ElevatedButton(
            onPressed: () {
              _selectDate(context);
            },
            child: Text("Choose Date"),
          ),
          Text("${selectedDate.day}/${selectedDate.month}/${selectedDate.year}")
        ],
      ),
    ),
  );
}
```

As the UI is ready now let's create the method to display DatePicker. We will be using the DateTime variable 'selectedDate' created above for **initialDate**. We are assigning the selected date to the variable in **setState()** to update the UI.

```
_selectDate(BuildContext context) async {
  final DateTime selected = await showDatePicker(
    context: context,
    initialDate: selectedDate,
    firstDate: DateTime(2010),
    lastDate: DateTime(2025),

  );
  if (selected != null && selected != selectedDate)
    setState(() {
      selectedDate = selected;
    });
}
```

➤ Example: Datepicker

Below is the complete example for where we create a datepicker and show it using showDatePicker function provided by flutter.

```
import 'package:flutter/material.dart';
void main()
{
  runApp(Datepicker_Roq_Ex());
}
class Datepicker_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Datepicker Roq App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  @override
  _MyHomePageState createState()
  {
    return _MyHomePageState();
  }
}

class _MyHomePageState extends State<MyHomePage> {
  String date = "";
  DateTime selectedDate = DateTime.now();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Datepicker Roq App"),
      ),
    );
  }
}
```

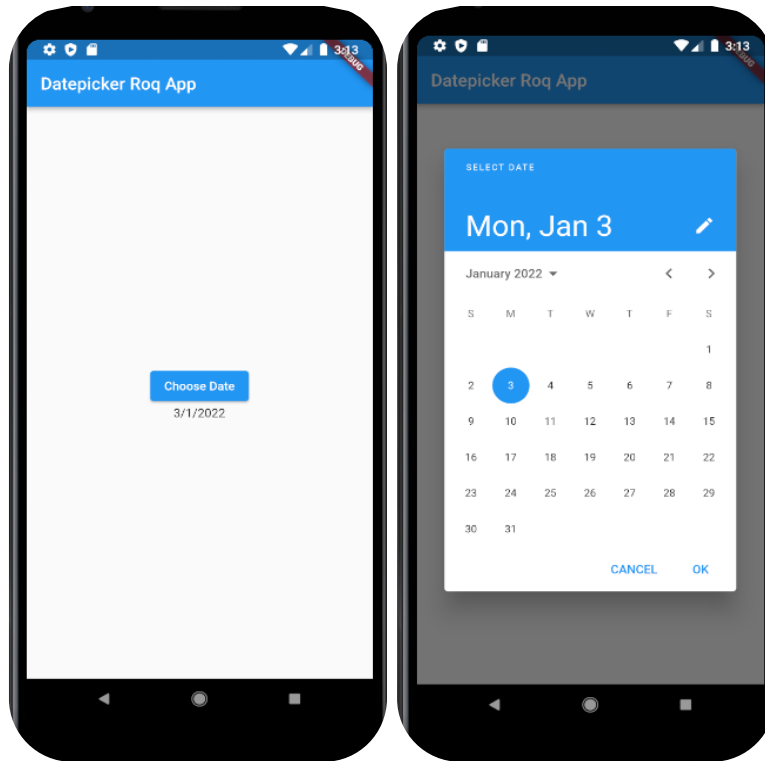
```

    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          ElevatedButton(
            onPressed: () {
              _selectDate(context);
            },
            child: Text("Choose Date"),
          ),
          Text("${selectedDate.day}/${selectedDate.month}/${selectedDate.year}")
        ],
      ),
    ),
  );
}

_selectDate(BuildContext context) async {
  final DateTime? selected = await showDatePicker(
    context: context,
    initialDate: selectedDate,
    firstDate: DateTime(2010),
    lastDate: DateTime(2025),
  );
  if (selected != null && selected != selectedDate)
    setState(() {
      selectedDate = selected;
    });
}
}

```

Output : When we run the app in Android Studio, we can see the following first screen in our Emulator. After clicking the button we can see the second screen as the following:



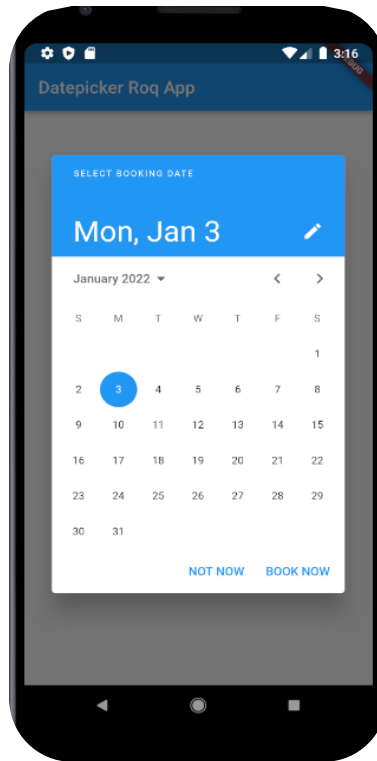
How to change help text, cancel and ok buttons text of datepicker

We can change the help text, cancel and ok buttons text of datepicker. we can do so by using the `helpText`, `cancelText` and `confirmText` properties. Let's see the code below.

```
showDatePicker(
  context: context,
  initialDate: selectedDate,
  firstDate: DateTime(2010),
  lastDate: DateTime(2025),
  //How to change help text, cancel and ok buttons text of datepicker
  helpText: "SELECT BOOKING DATE",
  cancelText: "NOT NOW",
  confirmText: "BOOK NOW"
);
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.

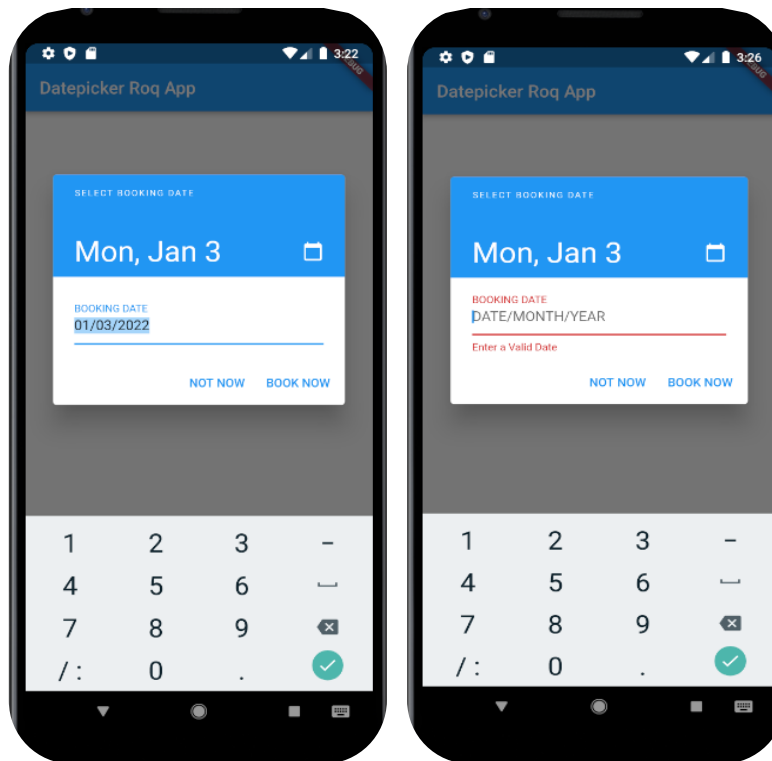


How to change label and hint text of a datepicker

```
showDatePicker(
  context: context,
  initialDate: selectedDate,
  firstDate: DateTime(2010),
  lastDate: DateTime(2025),
  //How to change help text, cancel and ok buttons text of datepicker
  helpText: "SELECT BOOKING DATE",
  cancelText: "NOT NOW",
  confirmText: "BOOK NOW",
  //How to change label and hint text of a datepicker
  fieldHintText: "DATE/MONTH/YEAR",
  fieldLabelText: "BOOKING DATE",
);
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



How to change format error and invalid error text

```
showDatePicker(
  context: context,
  initialDate: selectedDate,
  firstDate: DateTime(2010),
  lastDate: DateTime(2025),
  //How to change help text, cancel and ok buttons text of datepicker
  helpText: "SELECT BOOKING DATE",
  cancelText: "NOT NOW",
  confirmText: "BOOK NOW",
  //How to change label and hint text of a datepicker
  fieldHintText: "DATE/MONTH/YEAR",
  fieldLabelText: "BOOKING DATE ",
  //How to change format error and invalid error text
  errorFormatText: "Enter a Valid Date",
  errorInvalidText: "Date Out of Range"
);
```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator

How to display year first in datepicker

```
showDatePicker(
  context: context,
  initialDate: selectedDate,
  firstDate: DateTime(2010),
  lastDate: DateTime(2025),
  //How to change help text, cancel and ok buttons text of datepicker
  helpText: "SELECT BOOKING DATE",
  cancelText: "NOT NOW",
```

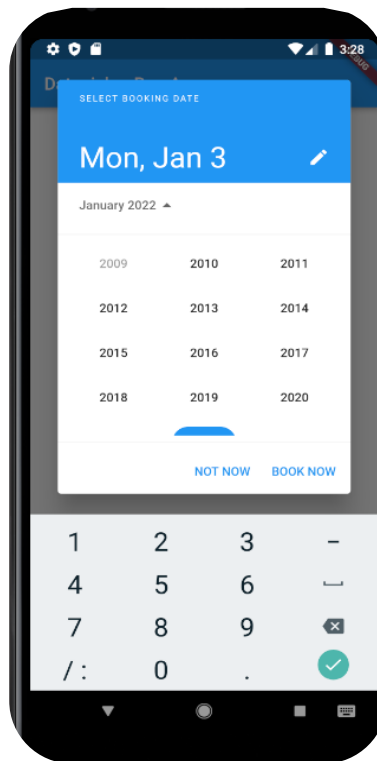
```

confirmText: "BOOK NOW",
//How to change label and hint text of a datepicker
fieldHintText: "DATE/MONTH/YEAR",
fieldLabelText: "BOOKING DATE ",
//How to change format error and invalid error text
errorFormatText: "Enter a Valid Date",
errorInvalidText: "Date Out of Range",
//How to display year first in datepicker
initialDatePickerMode: DatePickerMode.year
);

```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.



How to show a Text Input instead of calendar in datepicker

We can display a text input to enter the date manually instead of displaying a calendar. We can achieve this by setting the `initialEntryMode` property with `DatePickerEntryMode.input`. Let's see the code and output.

```

showDatePicker(
  context: context,
  initialDate: selectedDate,
  firstDate: DateTime(2010),
  lastDate: DateTime(2025),
  initialEntryMode: DatePickerEntryMode.input
);

```

Output

When we run the app in Android Studio, we can see the following screen in our Emulator.

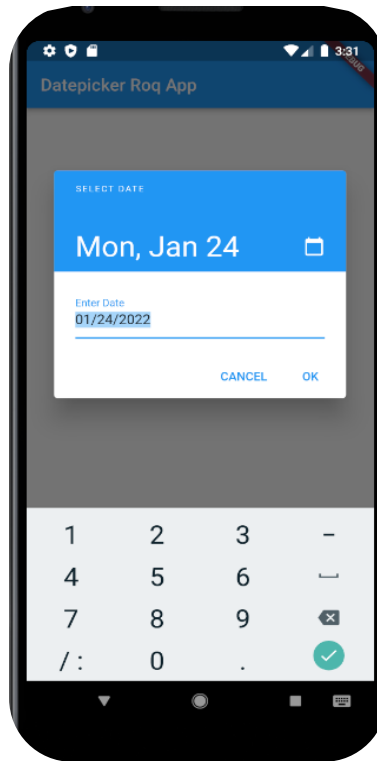


image picker

An image picker is one of the most widely used components in any app. Many popular apps, such as Facebook, Twitter, Instagram, WhatsApp, etc., have an image picker that enables users to select files from their device to use as a profile picture or share with their friends.

The most common use case for an image picker in a mobile app is to set an avatar for the user profile. In this tutorial, we'll show you how to create an image picker in Flutter. We'll build an example Flutter app that enables the user to either select a photo from the gallery or take a photo from the device's camera.

Image Picker Flutter - Pick Image from Gallery or Camera

So let's start to make these examples of images pickers so just follow these step and make it. Let's open your project if you have otherwise created a new flutter project. And Open **pubspec.yaml** then add the below lines.

- **Step 1:** Open **pubspec.yaml** and add a dependency for the image picker.
- **Step2:** Configuration for Android and iOS.
- **Step3:** Make UI and Implement it.

add Image Picker flutter library.

A Flutter plugin library for both Android and iOS that is been used to pick an image from a mobile phone gallery or camera. let's straight away start to implement the flutter image picker in our flutter app.

Step 1: Adding image picker dependency.

Once you have created your Flutter project or open an existing flutter project to implement an image picker flutter library. Then you need to add image picker dependencies,

Open **pubspec.yaml** then add the below lines

```
dependencies:  
  image_picker: ^0.8.6
```

Step 2: Configuration for Android and iOS

➤ Android Image Picker Configuration

When it comes to android there is no configuration required – the plugin should work out of the box. Just we need to set **request Legacy External Storage** to **true** Android > app > src > main > AndroidManifest.xml

```
<application  
  android:label="pick_images_camera_and_gallery"  
  android:requestLegacyExternalStorage="true"  
  android:icon="@mipmap/ic_launcher">
```

➤ IOS Image Picker Configuration

Add the following keys to your *Info.plist* file, located in

Project > ios > Runner > Info.plist

Copy-paste the below permission in **info.plist**. This app requires read and write permission Photo Library is used to ask uses to give permission for accessing the photo library. This is Called Privacy – Photo Library Usage Description. and Used to take photos from the device camera itself. This app requires to get access to micro phone

```
<key>NSPhotoLibraryUsageDescription</key>
```

```
<key>NSCameraUsageDescription</key>
```

```
<key>NSMicrophoneUsageDescription</key>
```

Step3: Explanation of flutter image picker.

here we make UI and implementation of camera and gallery to pick images and show them on. So least makes it open you main .dart file if you make it main file other you can create a new dart file and use the below code. hare full for how do you use camera and gallery in a flutter

➤ Example:

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
class ImagePicker_Roq_Ex extends StatefulWidget{
  @override
  State<StatefulWidget> createState() {
    // TODO: implement createState
    return CameraWidgetState();
  }
}
class CameraWidgetState extends State<ImagePicker_Roq_Ex>{
  PickedFile? imageFile_roq=null;
  Future<void> _showChoiceDialog(BuildContext context)
  {
    return showDialog(context: context,builder: (BuildContext context){

      return AlertDialog(
        title: Text("Choose option",style: TextStyle(color:
          Colors.blue),),
        content: SingleChildScrollView(
          child: Column(
            children: [
              Divider(height: 1,color: Colors.blue,),
              ListTile(
                onTap: (){
                  _openGallery(context);
                },
                title: Text("Gallery"),
                leading: Icon(Icons.account_box,color: Colors.blue,),
              ),
              Divider(height: 1,color: Colors.blue,),
              ListTile(
                onTap: (){
                  _openCamera(context);
                },
                title: Text("Camera"),
                leading: Icon(Icons.camera,color: Colors.blue,),
              ),
            ],
          ),
        ),
      );
    });
  }
  @override
  Widget build(BuildContext context) {
    // TODO: implement build
    return Scaffold(
      appBar: AppBar(
        title: Text("ImagePicker Roq App"),
        backgroundColor: Colors.green,
      ),
      body: Center(
        child: Container(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.spaceAround,
            children: [
              Card(
                child: ( imageFile_roq==null)?Text("No Image selected plz choose Image ")
```

```

: Image.file( File(imageFile_roq!.path)),
    ),
    MaterialButton(
      textColor: Colors.white,
      color: Colors.pink,
      onPressed: () {
        _showChoiceDialog(context);
      },
      child: Text("Select Image to Show"),
    ),
  ], ), ), ), ), ); }

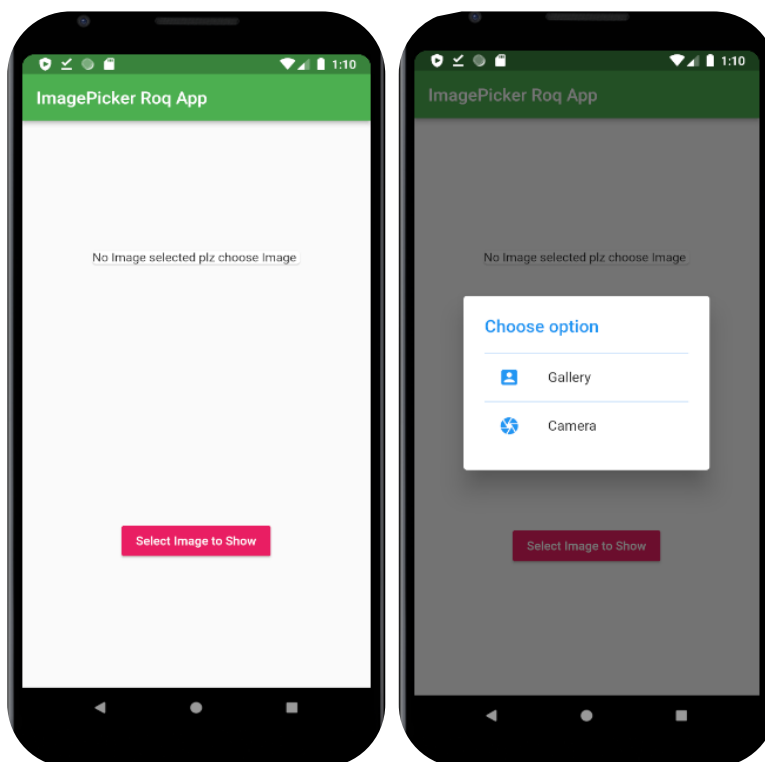
void _openGallery(BuildContext context) async{
  final pickedFile = await ImagePicker().getImage(
    source: ImageSource.gallery ,
  );
  setState(() {
    imageFile_roq = pickedFile!;
  });

  Navigator.pop(context);
}

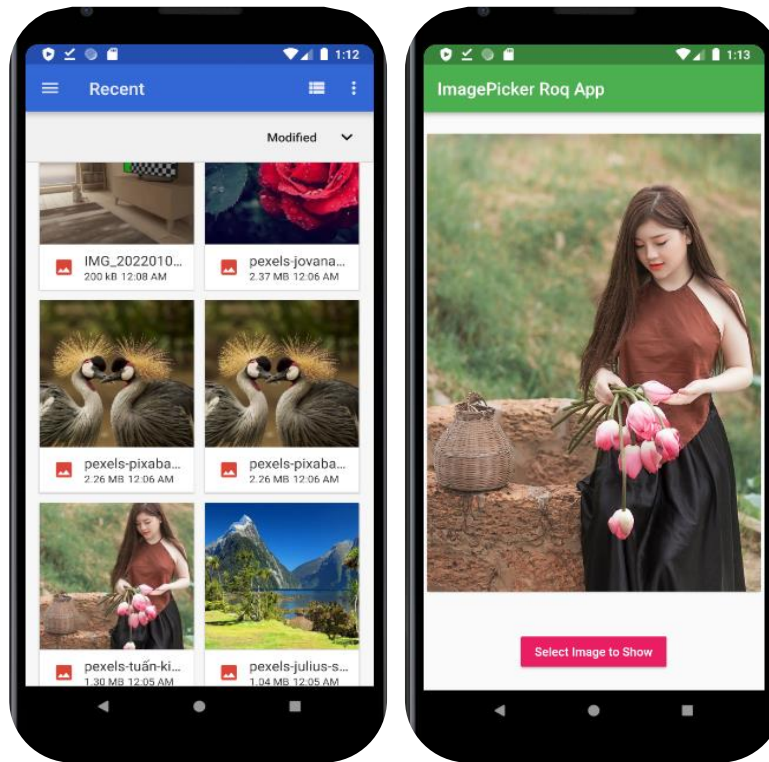
void _openCamera(BuildContext context) async{
  final pickedFile = await ImagePicker().getImage(
    source: ImageSource.camera ,
  );
  setState(() {
    imageFile_roq = pickedFile!;
  });
  Navigator.pop(context);
}
}

```

Output Run the app and you should see something like this.



Pick image from gallery:



Pick image from camera:

